Sri Sivasubramaniya Nadar College of Engineering, Chennai

(An autonomous Institution affiliated to Anna University)

Degree & Branch: M. Tech (Integrated) Computer Science & Engineering
Semester: V
Subject Code & Name: ICS1512 – Machine Learning Algorithms Laboratory
Academic year: 2025–2026 (Odd)
Batch: 2023-2028
Experiment: 3 – Ensemble Prediction and Decision Tree Model Evaluation

---

- NAME : A.S.Tritthik Thilagar
- Reg No: 3122237001057

Aim and Objective

To build classifiers such as Decision Tree, AdaBoost, Gradient Boosting, XGBoost, Random Forest, and Stacked Models (using SVM, Na¨ıve Bayes, Decision Tree) and evaluate their performance through 5-Fold Cross-Validation and hyperparameter tuning

Libraries Used

- pandas
- numpy
- matplotlib
- seaborn
- scikit-learn (sklearn)

Code

1. Importing Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
from sklearn.model_selection import train_test_split,
    cross_val_score, StratifiedKFold
from sklearn.preprocessing import StandardScaler,
    MinMaxScaler
from sklearn.metrics import accuracy_score, precision_score,
    recall_score, f1_score, classification_report, roc_curve,
    auc, confusion_matrix
from sklearn.linear_model import LogisticRegression, Ridge,
    Lasso
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.naive_bayes import GaussianNB, MultinomialNB,
    BernoulliNB
from sklearn.neighbors import KNeighborsClassifier
```

2. Importing Dataset and Preprocessing

```
drive.mount('/content/drive')

# Path to Wisconsin Diagnostic dataset
train_path = '/content/drive/MyDrive/Colab Notebooks/EX4/
    dataset_a4/wdbc.data'

# Dataset has no header, so we assign columns manually
cols = ['ID', 'Diagnosis'] + [f'feature_{i}' for i in range
    (1, 31)]
df = pd.read_csv(train_path, header=None, names=cols)

# Target encoding: M = 1 (Malignant), B = 0 (Benign)
df['Diagnosis'] = df['Diagnosis'].map({'M': 1, 'B': 0})

X = df.drop(columns=['ID', 'Diagnosis'])
y = df['Diagnosis']

numerical_cols = X.select_dtypes(include=np.number).columns.
    tolist()


for col in numerical_cols:
    Q1 = X[col].quantile(0.25)
    Q3 = X[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    X[col] = np.where(X[col] < lower_bound, lower_bound, X[
        col])
    X[col] = np.where(X[col] > upper_bound, upper_bound, X[
        col])
print("Outliers managed.")
```

```
28
29 X_train_full, X_test, y_train_full, y_test = train_test_split
   (
30     X, y, test_size=0.2, random_state=42, stratify=y
31 )
32 X_train, X_val, y_train, y_val = train_test_split(
33     X_train_full, y_train_full, test_size=0.25, random_state
          =42, stratify=y_train_full
34 )
```

3. Scaling Utility Functions

```
1  def choose_scaler(model):
2      if isinstance(model, (GaussianNB, KNeighborsClassifier,
          SVC)):
3          return StandardScaler()
4      elif isinstance(model, (MultinomialNB, BernoulliNB)):
5          return MinMaxScaler()
6      else:
7          return None
8
9  def auto_scale_model_data(model, X_train, X_test, X_val,
      X_train_full, numerical_cols):
10     scaler = choose_scaler(model)
11     if scaler is None:
12         return X_train.copy(), X_test.copy(), X_val.copy(),
              X_train_full.copy()
13     X_train_scaled = X_train.copy()
14     X_test_scaled = X_test.copy()
15     X_val_scaled = X_val.copy()
16     X_train_full_scaled = X_train_full.copy()
17     X_train_scaled[numerical_cols] = scaler.fit_transform(
          X_train[numerical_cols])
18     X_test_scaled[numerical_cols] = scaler.transform(X_test[
          numerical_cols])
19     X_val_scaled[numerical_cols] = scaler.transform(X_val[
          numerical_cols])
20     X_train_full_scaled[numerical_cols] = scaler.transform(
          X_train_full[numerical_cols])
21     return X_train_scaled, X_test_scaled, X_val_scaled,
          X_train_full_scaled
```

4. Decision Tree

```
1  from sklearn.model_selection import GridSearchCV,
      StratifiedKFold
2
3  # Base model
```

```
model_dt = DecisionTreeClassifier(random_state=42)

# Define hyperparameter grid
param_grid = {
    'criterion': ['gini', 'entropy', 'log_loss'],
    'max_depth': [None, 5, 10, 20, 30],
    'min_samples_split': [2, 5, 10, 20],
    'min_samples_leaf': [1, 2, 5, 10],
    'max_features': [None, 'sqrt', 'log2']
}

# Cross-validation strategy
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Grid Search
grid_search = GridSearchCV(
    estimator=model_dt,
    param_grid=param_grid,
    cv=cv,
    scoring='accuracy',
    n_jobs=-1,
    verbose=1
)

# Scale data (same as before)
X_train_scaled, X_test_scaled, X_val_scaled, X_train_full_scaled = auto_scale_model_data(
    model_dt, X_train, X_test, X_val, X_train_full, numerical_cols
)

# Fit GridSearch
grid_search.fit(X_train_full_scaled, y_train_full)

# Results of all parameter combinations
cv_results = pd.DataFrame(grid_search.cv_results_)

# Sort by mean test score
cv_results = cv_results.sort_values(by="mean_test_score", ascending=False)

# Show top results
# Show top results correctly for Decision Tree
print("\n--- All Grid Search Results (sorted by mean test score) ---")
print(cv_results[[
    "param_criterion",
    "param_max_depth",
    "param_min_samples_split",
    "param_min_samples_leaf",
```

```
    "param_max_features",
    "mean_test_score",
    "std_test_score",
    "rank_test_score"
]].head(10))



# Best params and score
print("\n--- Grid Search Results (Decision Tree) ---")
print("Best Parameters:", grid_search.best_params_)
print("Best Cross-Validation Accuracy:", grid_search.
    best_score_)

# Get best model
best_dt = grid_search.best_estimator_

# Evaluate with additional metrics using CV
cv_accuracy = cross_val_score(best_dt, X_train_full_scaled,
    y_train_full, cv=cv, scoring='accuracy')
cv_precision = cross_val_score(best_dt, X_train_full_scaled,
    y_train_full, cv=cv, scoring='precision')
cv_recall = cross_val_score(best_dt, X_train_full_scaled,
    y_train_full, cv=cv, scoring='recall')
cv_f1 = cross_val_score(best_dt, X_train_full_scaled,
    y_train_full, cv=cv, scoring='f1')

print(f"Accuracy: {cv_accuracy.mean():.2f}    {cv_accuracy.
    std():.2f}")
print(f"Precision: {cv_precision.mean():.2f}    {cv_precision
    .std():.2f}")
print(f"Recall: {cv_recall.mean():.2f}    {cv_recall.std():.2
    f}")
print(f"F1 Score: {cv_f1.mean():.2f}    {cv_f1.std():.2f}")

# Final fit on best model
best_dt.fit(X_train_full_scaled, y_train_full)

# Predictions
y_pred = best_dt.predict(X_test_scaled)
y_pred_proba = best_dt.predict_proba(X_test_scaled)[:, 1]


# 6. PERFORMANCE EVALUATION
print("\n--- Model Performance on Test Set (Decision Tree)
    ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred):.2f}")
print(f"Recall: {recall_score(y_test, y_pred):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred):.2f}")
```

```
print("\nClassification Report:\n", classification_report(
    y_test, y_pred))

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC
    curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('Decision Tree ROC Curve')
plt.legend(loc="lower right")
plt.show()

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Benign', 'Malignant'],
            yticklabels=['Benign', 'Malignant'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - Decision Tree')
plt.show()
```

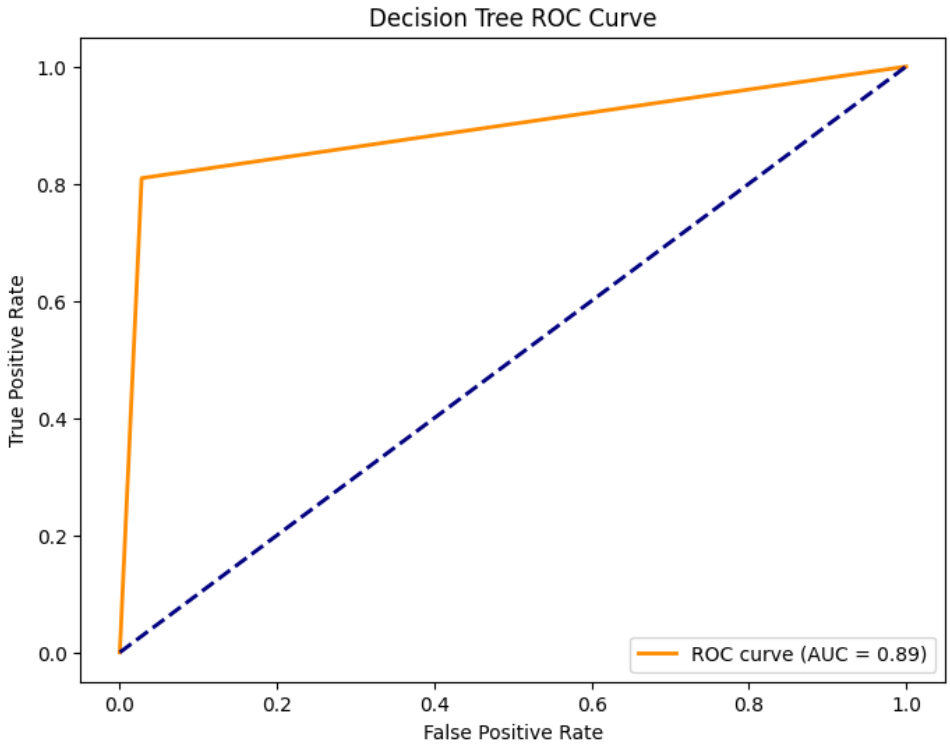

Figure 1: Decision Tree ROC curve

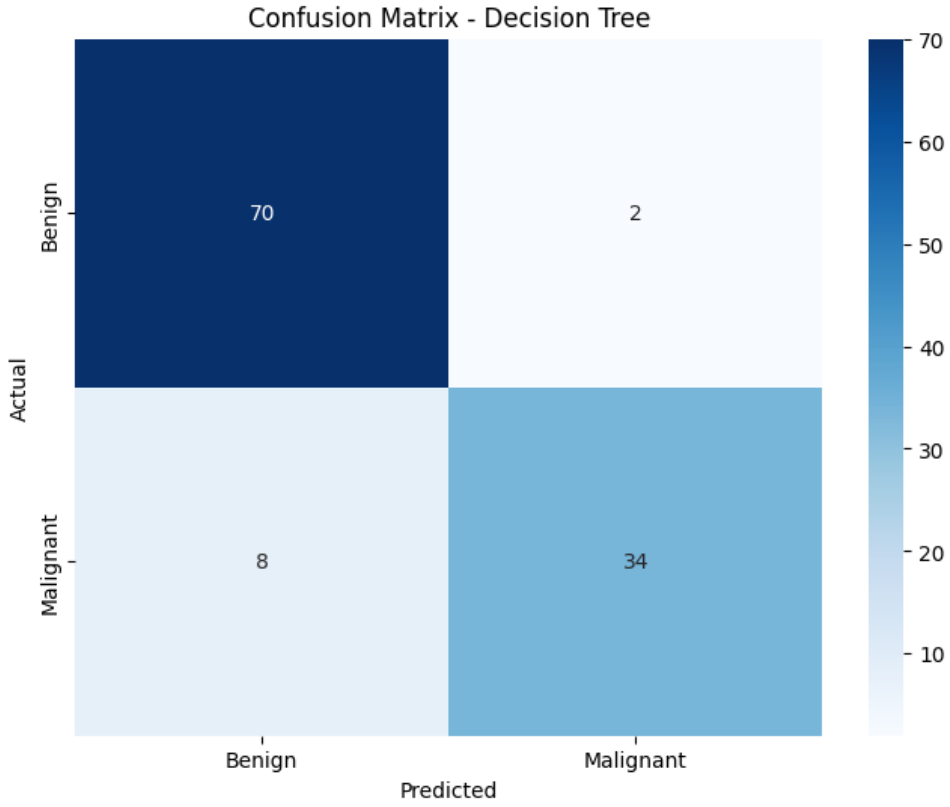

Figure 2: Decision Tree Confusion Matrix

5. AdaBoost Classifier

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier

# Base decision tree
base_tree = DecisionTreeClassifier(random_state=42)

# AdaBoost setup
model_ab = AdaBoostClassifier(estimator=base_tree,
    random_state=42)

# Parameter grid
param_grid_ab = {
    "n_estimators": [50, 100, 200],
    "learning_rate": [0.01, 0.1, 0.5, 1.0],
    "estimator__max_depth": [1, 2, 3]
}

grid_ab = GridSearchCV(model_ab, param_grid_ab, cv=5, scoring
    ="accuracy", n_jobs=-1, verbose=1)
grid_ab.fit(X_train_full, y_train_full)

# Results of all parameter combinations
cv_results = pd.DataFrame(grid_ab.cv_results_)

# Sort by mean test score
cv_results = cv_results.sort_values(by="mean_test_score",
    ascending=False)

# Show top results
print("\n--- All Grid Search Results (sorted by mean test
    score) ---")
print(cv_results[[
    "param_n_estimators",
    "param_learning_rate",
    "param_estimator__max_depth",
    "mean_test_score",
    "std_test_score",
    "rank_test_score"
]].head(10))



print("Best Params (AdaBoost):", grid_ab.best_params_)
best_ab = grid_ab.best_estimator_

# Predictions
```

```
y_pred = best_ab.predict(X_test)
y_pred_proba = best_ab.predict_proba(X_test)[:, 1]

# Evaluation
print("\n--- AdaBoost Performance ---")
print(classification_report(y_test, y_pred))


# --- PERFORMANCE EVALUATION ---
print("\n--- Model Performance on Test Set (AdaBoost + DecisionTree) ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred):.2f}")
print(f"Recall: {recall_score(y_test, y_pred):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred):.2f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('AdaBoost ROC Curve')
plt.legend(loc="lower right")
plt.show()

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Benign', 'Malignant'],
            yticklabels=['Benign', 'Malignant'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - AdaBoost')
plt.show()
```

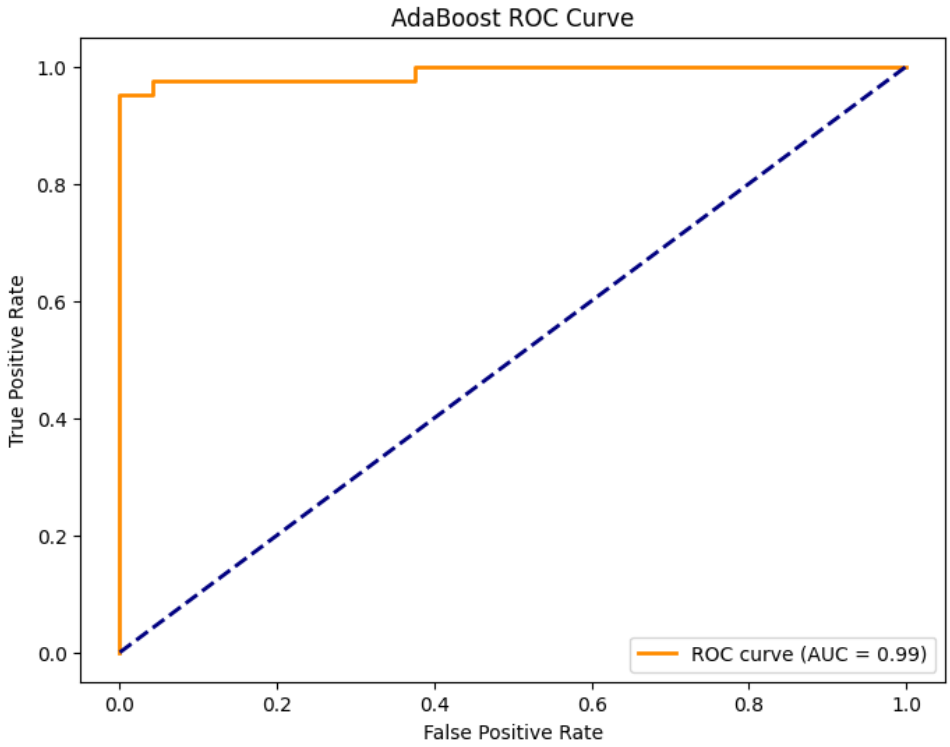

Figure 3: Adaboost Decision Tree Classifier ROC curve

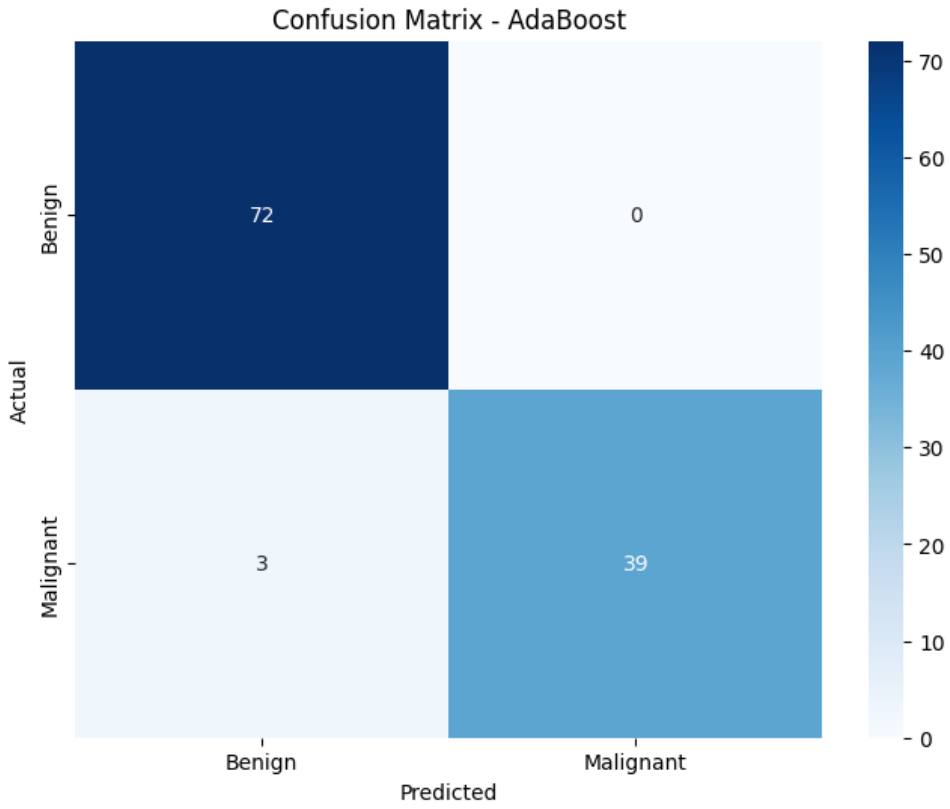

Figure 4: AdaBoost Decision Tree Classifier Confusion Matrix

6. Gradient Boosting

```
from sklearn.ensemble import GradientBoostingClassifier

from sklearn.ensemble import GradientBoostingClassifier

model_gb = GradientBoostingClassifier(random_state=42)

param_grid_gb = {
    "n_estimators": [100, 200, 300],
    "learning_rate": [0.01, 0.1, 0.2],
    "max_depth": [3, 5, 7],
    "subsample": [0.8, 1.0]
}

grid_gb = GridSearchCV(model_gb, param_grid_gb, cv=5, scoring
   ="accuracy", n_jobs=-1, verbose=1)
grid_gb.fit(X_train_full, y_train_full)



# Results of all parameter combinations
cv_results = pd.DataFrame(grid_gb.cv_results_)

# Sort by mean test score
cv_results = cv_results.sort_values(by="mean_test_score",
   ascending=False)

# Show top results
print(cv_results[[
    "param_n_estimators",
    "param_learning_rate",
    "param_max_depth",
    "param_subsample",
    "mean_test_score",
    "std_test_score",
    "rank_test_score"
]].head(10))



print("Best Params (GradientBoosting):", grid_gb.best_params_
   )
best_gb = grid_gb.best_estimator_

y_pred = best_gb.predict(X_test)
y_pred_proba = best_gb.predict_proba(X_test)[:, 1]

print("\n--- Gradient Boosting Performance ---")
print(classification_report(y_test, y_pred))

```

```
47
48 # --- PERFORMANCE EVALUATION ---
49 print("\n--- Model Performance on Test Set (Gradient Boosting
       ) ---")
50 print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
51 print(f"Precision: {precision_score(y_test, y_pred):.2f}")
52 print(f"Recall: {recall_score(y_test, y_pred):.2f}")
53 print(f"F1 Score: {f1_score(y_test, y_pred):.2f}")
54 print("\nClassification Report:\n", classification_report(
       y_test, y_pred))
55
56 # ROC Curve
57 fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
58 roc_auc = auc(fpr, tpr)
59
60 plt.figure(figsize=(8, 6))
61 plt.plot(fpr, tpr, color='darkgreen', lw=2, label=f'ROC curve
       (AUC = {roc_auc:.2f})')
62 plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
63 plt.xlabel('False Positive Rate')
64 plt.ylabel('True Positive Rate')
65 plt.title('Gradient Boosting ROC Curve')
66 plt.legend(loc="lower right")
67 plt.show()
68
69 # Confusion Matrix
70 cm = confusion_matrix(y_test, y_pred)
71 plt.figure(figsize=(8, 6))
72 sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
73             xticklabels=['Benign', 'Malignant'],
74             yticklabels=['Benign', 'Malignant'])
75 plt.xlabel('Predicted')
76 plt.ylabel('Actual')
77 plt.title('Confusion Matrix - Gradient Boosting')
78 plt.show()
```

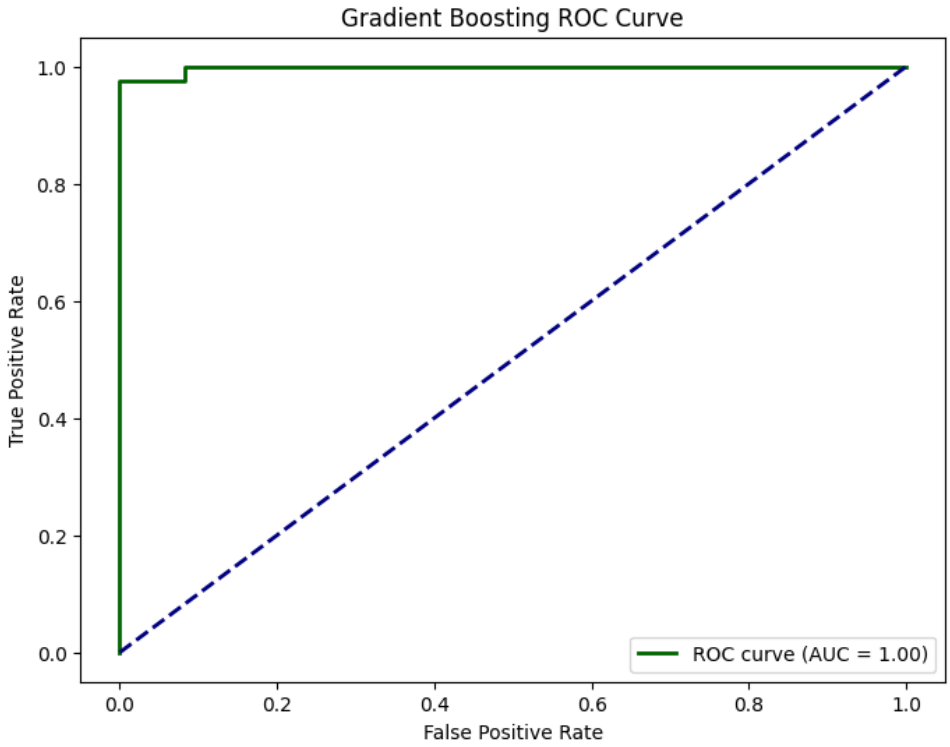

Figure 5: Gradient Boost ROC curve

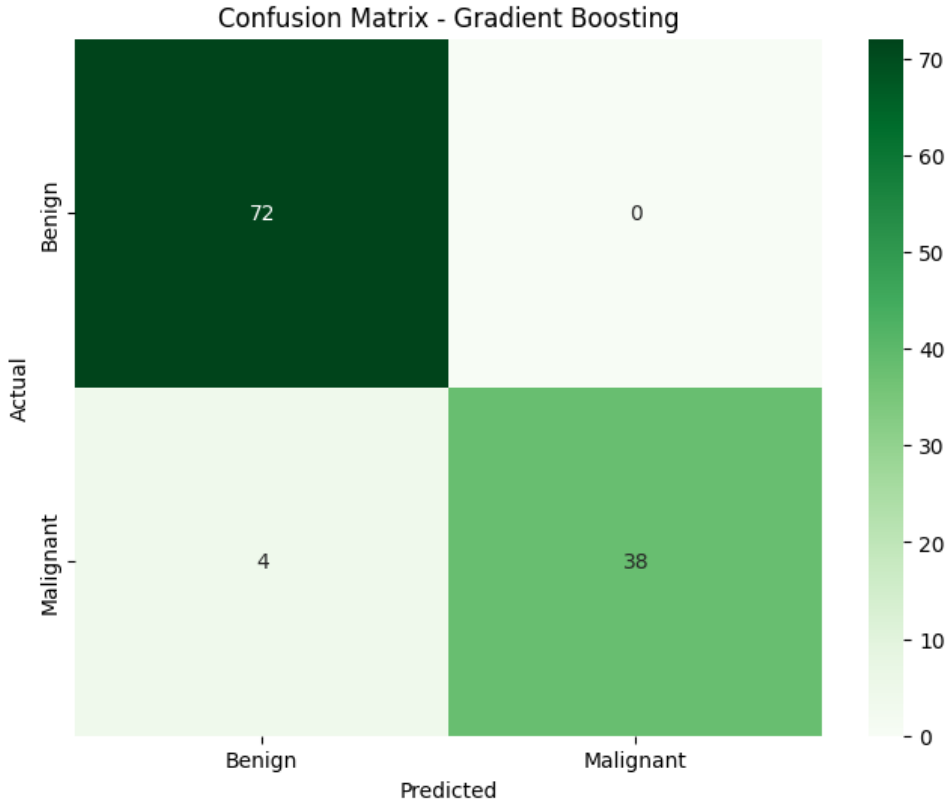

Figure 6: Gradient Boost Confusion Matrix

7. XG Boost

```
from xgboost import XGBClassifier

from xgboost import XGBClassifier

model_xgb = XGBClassifier(use_label_encoder=False,
    eval_metric="logloss", random_state=42)

param_grid_xgb = {
    "n_estimators": [100, 200, 300],
    "learning_rate": [0.01, 0.1, 0.2],
    "max_depth": [3, 5, 7],
    "gamma": [0, 1, 5],
    "subsample": [0.8, 1.0],
    "colsample_bytree": [0.8, 1.0]
}

grid_xgb = GridSearchCV(model_xgb, param_grid_xgb, cv=5,
    scoring="accuracy", n_jobs=-1, verbose=1)
grid_xgb.fit(X_train_full, y_train_full)

# Results of all parameter combinations
cv_results = pd.DataFrame(grid_xgb.cv_results_)

# Sort by mean test score
cv_results = cv_results.sort_values(by="mean_test_score",
    ascending=False)

print("\n--- All Grid Search Results (sorted by mean test
    score) ---")
print(cv_results[[
    "param_n_estimators",
    "param_learning_rate",
    "param_max_depth",
    "param_gamma",
    "param_subsample",
    "param_colsample_bytree",
    "mean_test_score",
    "std_test_score",
    "rank_test_score"
]].head(10))



print("Best Params (XGBoost):", grid_xgb.best_params_)
best_xgb = grid_xgb.best_estimator_

y_pred = best_xgb.predict(X_test)
y_pred_proba = best_xgb.predict_proba(X_test)[:, 1]

```

```
print("\n--- XGBoost Performance ---")
print(classification_report(y_test, y_pred))


# --- PERFORMANCE EVALUATION ---
print("\n--- Model Performance on Test Set (XG Boosting) ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print(f"Precision: {precision_score(y_test, y_pred):.2f}")
print(f"Recall: {recall_score(y_test, y_pred):.2f}")
print(f"F1 Score: {f1_score(y_test, y_pred):.2f}")
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# ROC Curve
fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
roc_auc = auc(fpr, tpr)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, color='darkgreen', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})')
plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('XG Boosting ROC Curve')
plt.legend(loc="lower right")
plt.show()

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
            xticklabels=['Benign', 'Malignant'],
            yticklabels=['Benign', 'Malignant'])
plt.xlabel('Predicted')
plt.ylabel('Actual')
plt.title('Confusion Matrix - XG Boosting')
plt.show()
```

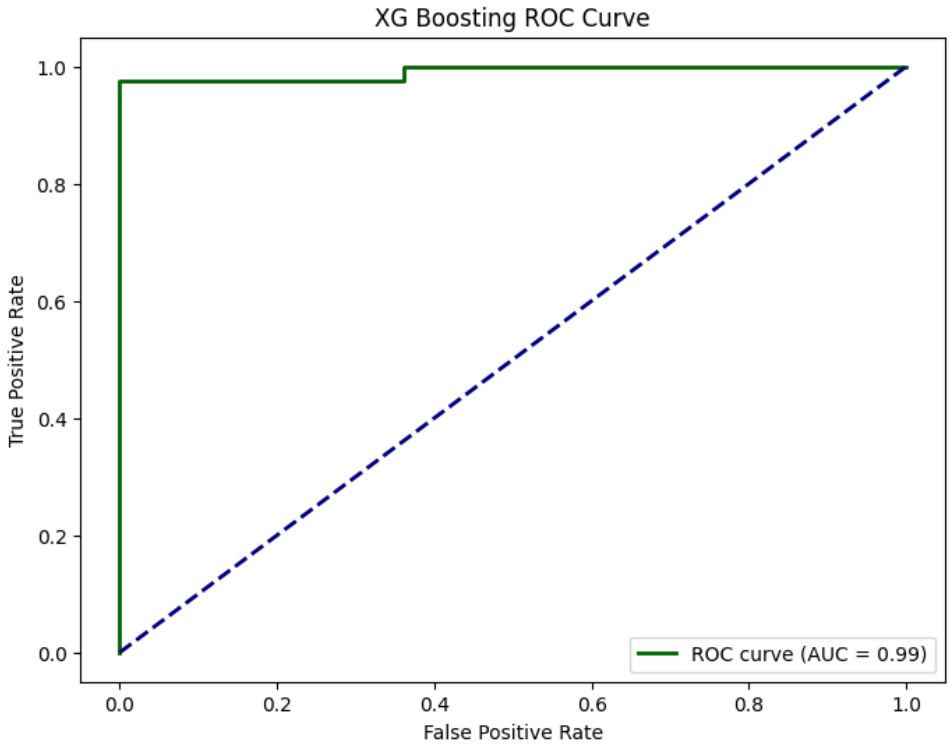

Figure 7: XG Boost ROC curve

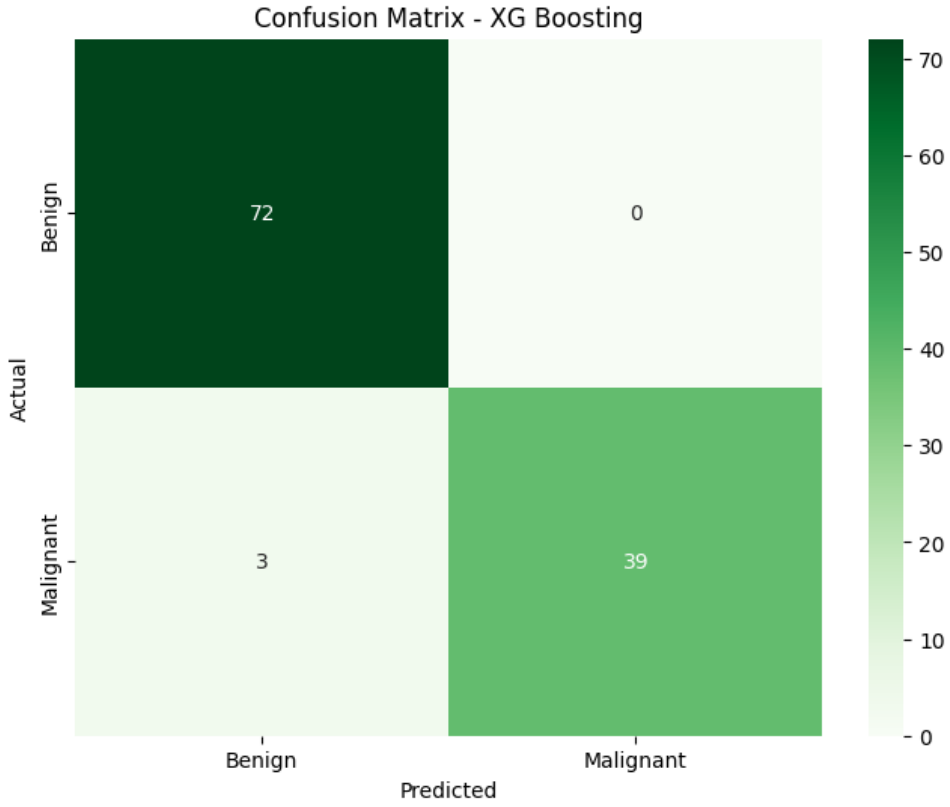

Figure 8: XG Boost Confusion Matrix

8. Random Forest Classifier

```
from sklearn.ensemble import RandomForestClassifier

from sklearn.ensemble import RandomForestClassifier

model_rf = RandomForestClassifier(random_state=42, n_jobs=-1)
param_grid_rf = {
    "n_estimators": [100, 200, 500],
    "max_depth": [None, 10, 20, 30],
    "criterion": ["gini", "entropy"],
    "max_features": ["sqrt", "log2"],
    "min_samples_split": [2, 5, 10]
}


grid_rf = GridSearchCV(model_rf, param_grid_rf, cv=5, scoring
   ="accuracy", n_jobs=-1, verbose=1)
grid_rf.fit(X_train_full, y_train_full)

# Results of all parameter combinations
cv_results = pd.DataFrame(grid_rf.cv_results_)

# Sort by mean test score
cv_results = cv_results.sort_values(by="mean_test_score",
   ascending=False)

# Show top results
print("\n--- All Grid Search Results (sorted by mean test
   score) ---")
print(cv_results[[
    "param_n_estimators",
    "param_max_depth",
    "param_criterion",
    "param_max_features",
    "param_min_samples_split",
    "mean_test_score",
    "std_test_score",
    "rank_test_score"
]].head(10))



print("Best Params (RandomForest):", grid_rf.best_params_)
best_rf = grid_rf.best_estimator_

y_pred = best_rf.predict(X_test)
y_pred_proba = best_rf.predict_proba(X_test)[:, 1]

print("\n--- Random Forest Performance ---")
print(classification_report(y_test, y_pred))
```

```
47
48
49  # --- PERFORMANCE EVALUATION ---
50  print("\n--- Model Performance on Test Set (Random Forest
        Classifer) ---")
51  print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
52  print(f"Precision: {precision_score(y_test, y_pred):.2f}")
53  print(f"Recall: {recall_score(y_test, y_pred):.2f}")
54  print(f"F1 Score: {f1_score(y_test, y_pred):.2f}")
55  print("\nClassification Report:\n", classification_report(
        y_test, y_pred))
56
57  # ROC Curve
58  fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)
59  roc_auc = auc(fpr, tpr)
60
61  plt.figure(figsize=(8, 6))
62  plt.plot(fpr, tpr, color='darkgreen', lw=2, label=f'ROC curve
        (AUC = {roc_auc:.2f})')
63  plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--')
64  plt.xlabel('False Positive Rate')
65  plt.ylabel('True Positive Rate')
66  plt.title('Random Forest Classifier ROC Curve')
67  plt.legend(loc="lower right")
68  plt.show()
69
70  # Confusion Matrix
71  cm = confusion_matrix(y_test, y_pred)
72  plt.figure(figsize=(8, 6))
73  sns.heatmap(cm, annot=True, fmt='d', cmap='Greens',
74              xticklabels=['Benign', 'Malignant'],
75              yticklabels=['Benign', 'Malignant'])
76  plt.xlabel('Predicted')
77  plt.ylabel('Actual')
78  plt.title('Confusion Matrix - Random Forest Classifier')
79  plt.show()
```

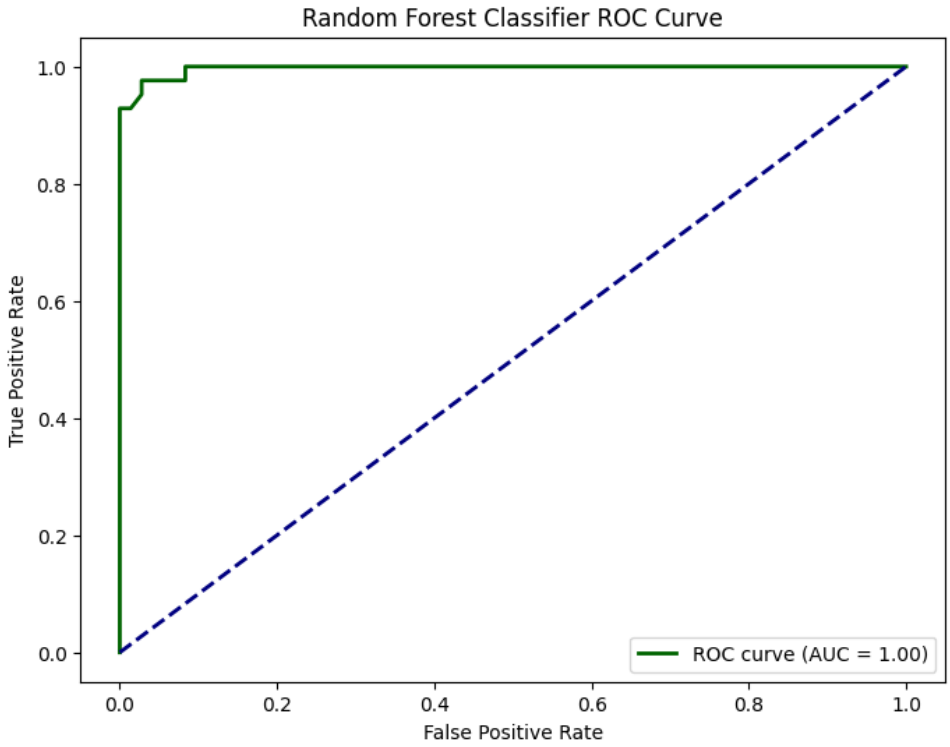

Figure 9: Random Forest ROC curve

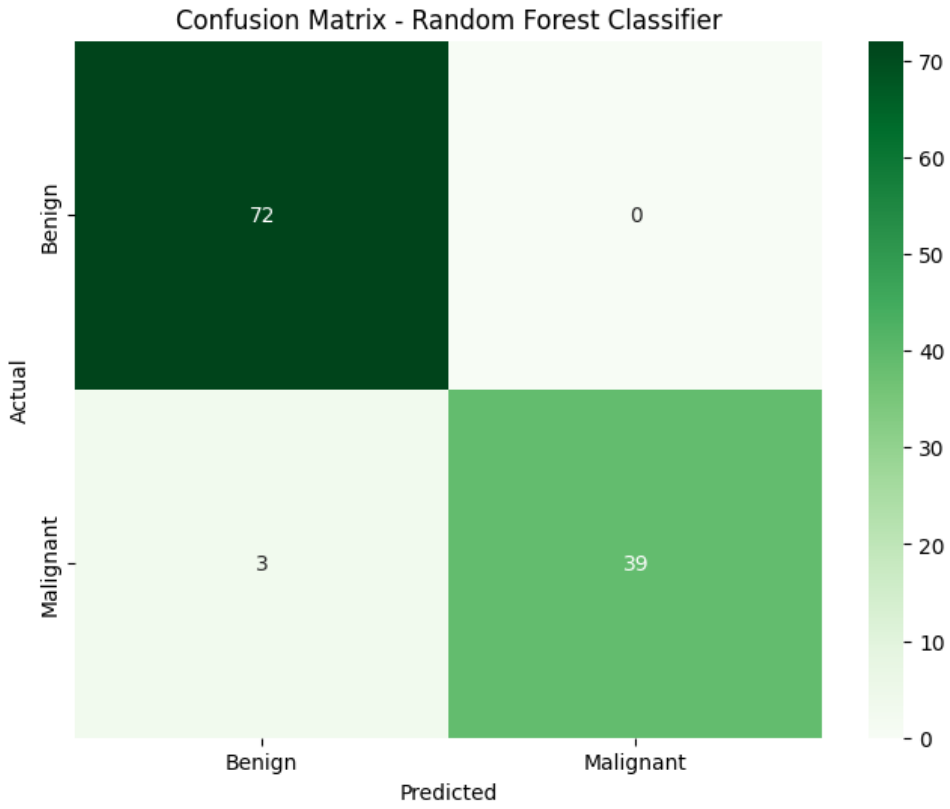

Figure 10: Random Forest Confusion Matrix

9. Stacked Model

```
model1 = StackingClassifier(
    estimators=[
        ("svm", make_pipeline(StandardScaler(), SVC(probability=
           True))),
        ("nb", GaussianNB()),
        ("dt", DecisionTreeClassifier(random_state=42))
    ],
    final_estimator=LogisticRegression(),
    cv=5,
    n_jobs=-1
)

# 2) SVM, Naive Bayes, Decision Tree     Random Forest
model2 = StackingClassifier(
    estimators=[
        ("svm", make_pipeline(StandardScaler(), SVC(probability=
           True))),
        ("nb", GaussianNB()),
        ("dt", DecisionTreeClassifier(random_state=42))
    ],
    final_estimator=RandomForestClassifier(random_state=42),
    cv=5,
    n_jobs=-1
)

# 3) SVM, Decision Tree, KNN     Logistic Regression
model3 = StackingClassifier(
    estimators=[
        ("svm", make_pipeline(StandardScaler(), SVC(probability=
           True))),
        ("dt", DecisionTreeClassifier(random_state=42)),
        ("knn", KNeighborsClassifier())
    ],
    final_estimator=LogisticRegression(),
    cv=5,
    n_jobs=-1
)

# ---------------------------
# TRAIN & EVALUATE MODELS
# ---------------------------

models = {
    "SVM+NB+DT     LogisticRegression": model1,
    "SVM+NB+DT     RandomForest": model2,
    "SVM+DT+KNN     LogisticRegression": model3
}

results = []
```

```
47
48  plt.figure(figsize=(8,6))
49
50  for name, model in models.items():
51      model.fit(X_train, y_train)
52      y_pred = model.predict(X_test)
53      y_proba = model.predict_proba(X_test)[:,1]    # for ROC
54
55      # Accuracy and F1
56      acc = accuracy_score(y_test, y_pred)
57      f1 = f1_score(y_test, y_pred)
58      results.append({"Model": name, "Accuracy": acc, "F1 Score":
            f1})
59
60      # ROC Curve
61      fpr, tpr, _ = roc_curve(y_test, y_proba)
62      roc_auc = auc(fpr, tpr)
63      plt.plot(fpr, tpr, lw=2, label=f"{name} (AUC = {roc_auc:.2f})
            ")
64
65      print(f"\n=== {name} ===")
66      print(classification_report(y_test, y_pred))
67      print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred)
            )
68
69  # ---------------------------
70  # ROC PLOT
71  # ---------------------------
72  plt.plot([0,1],[0,1], linestyle="--", color="gray")
73  plt.xlabel("False Positive Rate")
74  plt.ylabel("True Positive Rate")
75  plt.title("ROC Curve Comparison of Stacked Ensemble Models")
76  plt.legend(loc="lower right")
77  plt.grid(True)
78  plt.tight_layout()
79  plt.show()
80
81  # ---------------------------
82  # RESULTS TABLE
83  # ---------------------------
84  results_df = pd.DataFrame(results)
85  print("\nFinal Comparison Table:\n")
86  print(results_df)
```

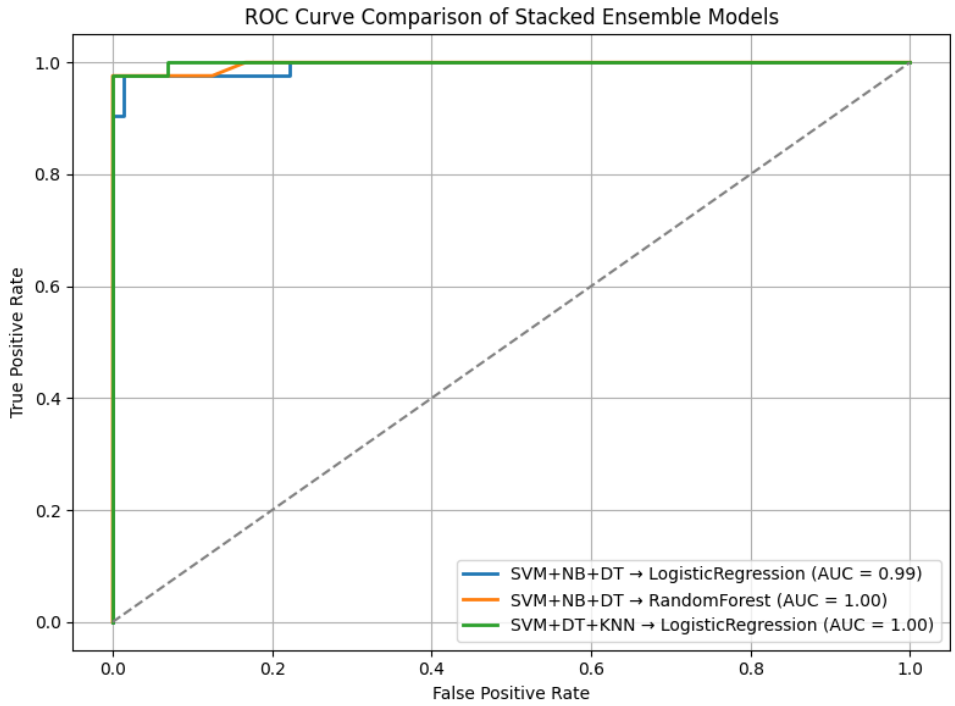

Figure 11: Stacked Model ROC curve

Results Tables

Table 1: Decision Tree Hyperparameter Tuning

Table 1: Decision Tree - Hyperparameter Tuning

criterion	max_depth	Accuracy	F1 Score
log_loss	30	0.938462	0.917143
log_loss	20	0.938462	0.916359
log_loss	20	0.938462	0.916359
entropy	10	0.938462	0.916359
entropy	10	0.938462	0.916359

Table 2: AdaBoost Classifier Hyperparameter Tuning

Table 2: AdaBoost - Hyperparameter Tuning

n_estimators	learning_rate	Accuracy	F1 Score
50	0.01	0.92967	0.90629
100	0.01	0.92967	0.90629
200	0.01	0.92967	0.90629
50	0.10	0.92967	0.90629
100	0.10	0.92967	0.90629

Table 3: Gradient Boost Hyperparameter Tuning

Table 3: Gradient Boosting - Hyperparameter Tuning

n_estimators	learning_rate	max_depth	Accuracy	F1 Score
300	0.2	3	0.958242	0.944482
200	0.2	3	0.956044	0.942046
300	0.1	3	0.956044	0.940641
100	0.2	3	0.953846	0.938690
100	0.1	3	0.953846	0.938236

Table 4: XGBoost Hyperparameter Tuning

Table 4: XGBoost - Hyperparameter Tuning

n_estimators	learning_rate	max_depth	gamma	Accuracy	F1 Score
300	0.2	7	0	0.964835	0.953270
200	0.2	7	0	0.964835	0.953270
300	0.1	7	0	0.964835	0.952766
300	0.1	5	0	0.964835	0.952766
100	0.1	7	0	0.962637	0.951118

Table 5: Random Forest Classifier Hyperparameter Tuning

Table 5: Random Forest - Hyperparameter Tuning

n_estimators	max_depth	criterion	Accuracy	F1 Score
200	10	entropy	0.964835	0.952111
200	20	entropy	0.964835	0.952111
200	30	entropy	0.964835	0.952111
200	None	entropy	0.964835	0.952111
500	30	entropy	0.962637	0.949126

Table 6: Stacked Ensemble Models

Table 6: Stacked Ensemble - Hyperparameter Tuning

Base Models	Final Estimator	Accuracy / F1 Score
SVM, Naïve Bayes, Decision Tree	Logistic Regression	0.973684 / 0.963855
SVM, Naïve Bayes, Decision Tree	Random Forest	0.982456 / 0.976190
SVM, Decision Tree, KNN	Logistic Regression	0.982456 / 0.975610

Learning Outcomes

- Learned how to build and tune Decision Tree classifiers, exploring the impact of parameters such as `criterion` and `max_depth`.

- Gained practical experience with ensemble methods (AdaBoost, Gradient Boosting, XGBoost, Random Forest), and understood how boosting and bagging improve prediction performance.
- Understood the role of hyperparameters like `n_estimators`, `learning_rate`, `max_depth`, and `gamma`, and their influence on bias-variance tradeoff.
- Practiced performing hyperparameter tuning via GridSearchCV and interpreting cross-validation results for fair model comparison.
- Developed the ability to evaluate and compare ensemble techniques using metrics such as accuracy and F1-score, and reported results systematically in tables.
- Learned how stacked ensemble models combine multiple base learners with a final estimator to enhance generalization, and compared their performance against individual models.
- Improved overall understanding of tree-based models and ensemble strategies, consolidating best practices for applying them to real-world datasets.